SQL Basic Operations

1. INSERT — Adding Data

The `INSERT` statement is used to add new records to a table.

```
-- Insert a single student
INSERT INTO students (id, name, age, course, marks)
VALUES (10, 'Rashid', 19, 'Data Science', 87);

-- Insert multiple students
INSERT INTO students (id, name, age, course, marks)
VALUES
(11, 'Akhil', 20, 'Python', 92),
(12, 'Neha', 18, 'MERN', 78);
```

---

2. SELECT — Retrieving Data

The `SELECT` statement is used to retrieve data from a table.

```
-- Select all columns
SELECT * FROM students;

-- Select specific columns
SELECT name, marks
FROM students;

-- Select name and course
SELECT name, course
FROM students;
```

---

3. WHERE — Filtering Data

The `WHERE` clause is used to filter records based on a condition.

```
-- Students who scored more than 80
```

```
SELECT * FROM students
WHERE marks > 80;

-- Students who are 18 years old
SELECT * FROM students
WHERE age = 18;
```

---

4. UPDATE — Modifying Data

The `UPDATE` statement is used to modify existing records.

```
-- Change Rashid's marks
UPDATE students
SET marks = 90
WHERE id = 10;

-- Change a student's course
UPDATE students
SET course = 'Machine Learning'
WHERE id = 11;
```

---

5. DELETE — Removing Data

The `DELETE` statement is used to remove records from a table.

```
-- Delete a specific student
DELETE FROM students
WHERE id = 10;

-- Delete students who scored below 40
DELETE FROM students
WHERE marks < 40;
```

Quick Summary

INSERT → Add new records
SELECT → Retrieve records

WHERE → Filter records
UPDATE → Modify records
DELETE → Remove records

Important Note

Always use `WHERE` with `UPDATE` and `DELETE` when you want to affect specific rows.

```
-- Without WHERE: affects ALL rows
UPDATE students
SET marks = 100;

-- With WHERE: affects only matching rows
UPDATE students
SET marks = 100
WHERE id = 10;
```

1. Introduction to JOINs

Suppose we have two tables:

`students`

```
id | name
---+--------
1  | Reema
2  | Shahana
3  | Neha
4  | Abhinov
```

`marks`

```
student_id | mark
-----------+-----
1          | 85
2          | 90
3          | 75
5          | 88
```

Notice:

- Student 1, 2, 3 exist in both tables.

- Student `4` exists only in `students`.
- Student `5` exists only in `marks`.

A **JOIN** allows us to combine related data from these tables.

The common relationship is:

students.id = marks.student_id

---

2. INNER JOIN

An `INNER JOIN` returns **only rows that exist in BOTH tables**.

```
SELECT students.name, marks.mark
FROM students
INNER JOIN marks
ON students.id = marks.student_id;
```

Result:

```
name     | mark
---------+-----
Reema    | 85
Shahana  | 90
Neha     | 75
```

Student `Abhinov` isn't included because there is no matching record in `marks`.

Student `5` isn't included because there is no matching student.

Think:

Table A ∩ Table B

ONLY MATCHING DATA

---

3. LEFT JOIN

A `LEFT JOIN` returns:

> **Everything from the LEFT table + matching data from the RIGHT table.**

```
SELECT students.name, marks.mark
FROM students
LEFT JOIN marks
ON students.id = marks.student_id;
```

Result:

```
name      | mark
----------+-----
Reema     | 85
Shahana   | 90
Neha      | 75
Abhinov   | NULL
```

Why is Abhinov included?

Because `students` is the **left table**, and a LEFT JOIN keeps **all rows from the left table**.

There is no matching mark, so PostgreSQL gives:

NULL

Think:

LEFT JOIN

ALL of A
+
matching B

---

4. RIGHT JOIN

A `RIGHT JOIN` does the opposite.

It keeps **everything from the RIGHT table**.

```
SELECT students.name, marks.mark
FROM students
RIGHT JOIN marks
ON students.id = marks.student_id;
```

Result:

```
name      | mark
----------+-----
Reema     | 85
Shahana   | 90
Neha      | 75
NULL      | 88
```

Why is the last row `NULL`?

Because `student_id = 5` exists in `marks`, but there is no student with `id = 5`.

Think:

RIGHT JOIN

matching A
+
ALL of B

---

5. FULL JOIN

A `FULL JOIN` keeps **everything from both tables**.

```
SELECT students.name, marks.mark
FROM students
FULL JOIN marks
ON students.id = marks.student_id;
```

Result:

```
name      | mark
----------+-----
Reema     | 85
Shahana   | 90
Neha      | 75
Abhinov   | NULL
NULL      | 88
```

It includes:

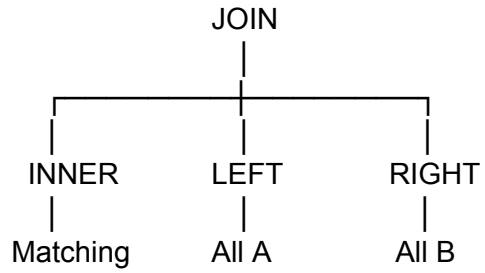
- Matching records
- Students without marks
- Marks without students

Think:

FULL JOIN

EVERYTHING from A
+
EVERYTHING from B

The easiest way to remember



And:

FULL JOIN = EVERYTHING

Visual idea

INNER JOIN
  A ∩ B

LEFT JOIN
  A + matching B

RIGHT JOIN
  matching A + B

FULL JOIN
  A + B

⭐ Important SQL syntax

The general structure is:

```
SELECT columns
FROM table1
JOIN table2
ON table1.column = table2.column;
```

For example:

```
SELECT s.name, m.mark
FROM students AS s
INNER JOIN marks AS m
ON s.id = m.student_id;
```

Here:

s → students
m → marks

These are called **aliases** and make JOIN queries much easier to write.

Practice with your own tables

You can create these two tables in your `sample` database:

```
CREATE TABLE students (
    id INT,
    name VARCHAR(50)
);

CREATE TABLE marks (
    student_id INT,
    mark INT
);
```

Insert:

```
INSERT INTO students VALUES
(1, 'Reema'),
(2, 'Shahana'),
(3, 'Neha'),
(4, 'Abhinov');
```

And:

```
INSERT INTO marks VALUES
(1, 85),
(2, 90),
(3, 75),
(5, 88);
```

Then practice:

```
SELECT *
FROM students
INNER JOIN marks
ON students.id = marks.student_id;
```

SQL Subqueries — Notes

A **subquery** is a query written inside another SQL query.

Think of it as:

Outer query = main question
Subquery = helps answer the main question

Basic structure:

```
SELECT column
FROM table
WHERE column = (
   SELECT column
   FROM table
   WHERE condition
);
```

---

a) Introduction to Subqueries

A subquery is enclosed inside **parentheses** `( )`.

Example:

Find employees who earn more than the average salary.

First, find the average:

```
SELECT AVG(salary)
FROM employees;
```

Then use that result in another query:

```
SELECT *
FROM employees
WHERE salary > (
```

```
    SELECT AVG(salary)
    FROM employees
);
```

How SQL processes it

The subquery:

```
SELECT AVG(salary)
FROM employees;
```

might return:

85000

Then the outer query becomes conceptually:

```
SELECT *
FROM employees
WHERE salary > 85000;
```

Important

A subquery can be used in places such as:

WHERE

HAVING

FROM

SELECT

The most common beginner use is inside `WHERE`.

---

b) Single-Row Subqueries

A **single-row subquery returns exactly one value/row**.

For example:

SELECT MAX(salary)

FROM employees;

returns one value:

120000

You can use it with comparison operators:

=

>

<

>=

<=

<>

Example 1 — Highest salary

SELECT *

FROM employees

WHERE salary = (

```
    SELECT MAX(salary)
    FROM employees
);
```

Example 2 — Employees earning more than average

```
SELECT *
FROM employees
WHERE salary > (
    SELECT AVG(salary)
    FROM employees
);
```

Example 3 — Employees earning less than Vikram

```
SELECT *
FROM employees
WHERE salary < (
    SELECT salary
    FROM employees
    WHERE name = 'Vikram'
);
```

Here:

```
SELECT salary
FROM employees
```

WHERE name = 'Vikram';

must return **one value**.

⚠️ Important

This can cause an error:

```
SELECT *
FROM employees
WHERE salary = (
   SELECT salary
   FROM employees
);
```

because the subquery returns **multiple rows**, but = expects one value.

---

c) Multiple-Row Subqueries

A **multiple-row subquery returns more than one value**.

Example:

```
SELECT salary
FROM employees
WHERE department_id = 2;
```

might return:

72000

110000

52000

75000

You can't normally do:

```
WHERE salary = (
    SELECT salary
    FROM employees
    WHERE department_id = 2
);
```

because there are multiple results.

Instead, use operators designed for multiple values.

IN

```
SELECT *
FROM employees
WHERE salary IN (
    SELECT salary
    FROM employees
    WHERE department_id = 2
);
```

Meaning:

Find employees whose salary matches **any** salary returned by the subquery.

ANY

```
SELECT *
FROM employees
WHERE salary > ANY (
    SELECT salary
    FROM employees
    WHERE department_id = 2
);
```

Meaning:

Salary is greater than **at least one** value returned by the subquery.

If the subquery returns:

52000

72000

110000

A salary of `60000` qualifies because:

60000 > 52000 ✅

ALL

```
SELECT *
```

```
FROM employees
WHERE salary > ALL (
   SELECT salary
   FROM employees
   WHERE department_id = 2
);
```

Meaning:

> Salary must be greater than **every value** returned by the subquery.

For:

52000

72000

110000

the salary must be greater than `110000`.

`IN` vs `ANY` vs `ALL`

Operator	Meaning
`IN`	Matches one of the returned values
`ANY`	Condition is true for at least one value

`ALL`	Condition is true for every value

---

d) Nested Subqueries

A **nested subquery** is a subquery inside another subquery.

In other words:

Outer Query

↓

Subquery

↓

Another Subquery

Example

Find employees whose salary is greater than the **average salary of the department that has the highest average salary**.

```
SELECT *
FROM employees
WHERE salary > (
    SELECT AVG(salary)
    FROM employees
    WHERE department_id = (
        SELECT department_id
        FROM employees
```

```
        GROUP BY department_id
        ORDER BY AVG(salary) DESC
        LIMIT 1
    )
);
```

There are multiple levels:

Outer query

↓

Find average salary

↓

Find department with highest average

That's a nested subquery.

Another Important Type: Correlated Subquery

You should know this because it is a natural next step.

A **normal subquery** can execute independently:

```
SELECT *
FROM employees
WHERE salary > (
    SELECT AVG(salary)
```

```
    FROM employees
);
```

A **correlated subquery** depends on the current row of the outer query.

Example:

Find employees earning more than the average salary of their own department.

```
SELECT e.name, e.salary, e.department_id
FROM employees e
WHERE e.salary > (
    SELECT AVG(e2.salary)
    FROM employees e2
    WHERE e2.department_id = e.department_id
);
```

The inner query uses:

e.department_id

from the **outer query**.

So conceptually:

Employee 1

↓

Calculate average of Employee 1's department

↓

Compare salary

Employee 2

↓

Calculate average of Employee 2's department

↓

Compare salary

🧠 Subquery Cheat Sheet

Single value

Use:

=

>

<

>=

<=

<>

Example:

```
WHERE salary > (
   SELECT AVG(salary)
   FROM employees
);
```

Multiple values

Use:

IN

ANY

ALL

Example:

```
WHERE salary IN (
   SELECT salary
   FROM employees
   WHERE department_id = 2
);
```

Nested

Subquery inside another subquery:

```
SELECT ...
WHERE ... (
   SELECT ...
   WHERE ... (
      SELECT ...
   )
);
```

Correlated

Inner query refers to outer query:

```
SELECT ...
FROM employees e
WHERE salary > (
    SELECT AVG(...)
    FROM employees e2
    WHERE e2.department_id = e.department_id
);
```

The problem i had is confusing in syntax